

GIT INSIDE OUT

Dhruv Sunil

Department of Computer Science and Engineering
Manipal Institute of Technology, Manipal, India
dhruvsunil2708@gmail.com

ABSTRACT

Git is one of the popular distributed version control systems that is widely used for managing source code and collaborating on software projects. It allows developers to track changes made to files, maintain different versions of a project, work on multiple features simultaneously, and restore previous versions when required. Git is commonly used together with platforms such as GitHub, which provide remote repositories for storing and collaborating on Git projects. This paper provides a description on how Git works, explaining its purpose, basic working principles, repository structure, commonly used commands, branching, merging, rebasing, and interaction with remote repositories. The objective is to understand how Git fits into the software development workflow and why it is an important tool for computer science students and developers. *Keywords— Git, Version Control, Repository, GitHub, Commit, Branch, Merge, Rebase, Push, Pull, Distributed Version Control*

I. INTRODUCTION

When developing a software project, developers continuously make changes to their code. Managing these changes manually can become difficult, especially when a project contains many files or multiple people are working on it. For example, creating multiple copies such as `project_1`, `project_2`, and `project_final` makes it difficult to determine which version is correct.

Version control systems solve this problem by keeping track of changes made to a project. Git is one of the most widely used version control systems. It was originally created by Linus Torvalds in 2005 for the development of the Linux kernel.

Git allows developers to save snapshots of their project at different points in time. These snapshots are called commits. Developers can also create

branches to work on different features without affecting the main version of the project.

Git is a distributed version control system, meaning that each developer can have a complete copy of the repository on their own computer. This allows many Git operations to be performed without requiring a constant internet connection and is relatively better than centralized version control system as if the

server goes down no one can work on the project anymore which is not a problem in Git.

II. WHAT IS GIT? Git is a distributed version control system used to track changes in files and directories. It enables developers to maintain a complete history of project changes, create isolated branches for experimental features, and safely merge independent updates into a centralized codebase without data loss.

III. GITHUB

Git and GitHub are related but are not the same thing. Git is the version control software that runs on a computer, while GitHub is an online platform that hosts Git repositories and provides additional collaboration features.

A developer can use Git locally without using GitHub. However, GitHub is useful when code needs to be shared with other people or stored remotely.

IV. HOW GIT WORKS

Git mainly works by maintaining different areas through which changes move before becoming part of the project's history. The basic workflow is:

Working Directory → Staging Area → Local Repository → Remote Repository

The Working Directory contains the actual files that the developer is currently working on. The Staging Area is used to select which changes should be included in the next commit. The Local Repository stores committed project history on the developer's computer. A remote repository is a centralized server where developers upload and share their code commonly on a service such as GitHub.

V. INITIALIZING A GIT REPOSITORY

A normal project directory is not automatically a Git repository. To start using Git in a project, the command `git init` is used. This creates a hidden `.git` directory containing the information required by Git to track the project, including its history, branches, and configuration.

VI. IMPORTANT GIT COMMANDS

The following commands form a useful foundation for beginners learning Git.

Command	Purpose
<code>git init</code>	Creates a new Git repository

git status	Shows the current state of files
git add	Adds changes to the staging area
git commit	Saves staged changes to the repository
git log	Displays commit history
git branch	Lists or creates branches
git switch	Changes branches
git merge	Combines branches
git rebase	Reapplies commits on top of another base
git clone	Copies a remote repository locally
git push	Uploads local commits
git pull	Downloads and integrates remote changes
git fetch	Downloads remote information without merging
git remote	Manages remote repository connections

VII. COMMITS

A commit is like a snapshot. It represents a recorded version of the project's changes. A project may have a history such as: Commit 1: Created project; Commit 2: Added homepage; Commit 3: Added login system; Commit 4: Fixed login bug. This history allows developers to understand how the project developed over time. Git commits also contain information such as the author, timestamp, commit identifier, and relationship to previous commits. Committing too often or too less is bad. Committing too often makes debugging difficult while committing too less is a problem when u make an error and now you can't easily revert to an older version of your code.

```
dhruv@Ohrvus-MacBook-Air Git_Task1 % git commit -m "Added Task one solutions"
[main (root-commit) 72a2b5] Added Task one solutions
9 files changed, 272 insertions(+)
create mode 100644 FMT1Q1.py
create mode 100644 FMT1Q2.py
create mode 100644 FMT1Q3.py
create mode 100644 FMT1Q4.py
create mode 100644 FMT1Q5.py
create mode 100644 FMT1Q6.py
create mode 100644 FMT1Q7.py
create mode 100644 FMT1Q8.py
create mode 100644 README.md
```

VIII. TRACKED AND UNTRACKED FILES

Git distinguishes between files that it is already managing and files that it has not yet started tracking. A newly created file may appear as untracked when using git status. Now git doesn't track this file even after initializing. To track we need to send it to the staging area using: git add <file_name> or we can use: git add *.txt this adds

all txt files or: git add . the period means to add all files in repository.

```
dhruv@Ohrvus-MacBook-Air Git_Task1 % git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    FMT1Q1.py
    FMT1Q2.py
    FMT1Q3.py
    FMT1Q4.py
    FMT1Q5.py
    FMT1Q6.py
    FMT1Q7.py
    FMT1Q8.py
    README.md

nothing added to commit but untracked files present (use "git add" to track)

dhruv@Ohrvus-MacBook-Air Git_Task1 % git add .
dhruv@Ohrvus-MacBook-Air Git_Task1 % git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   FMT1Q1.py
    new file:   FMT1Q2.py
    new file:   FMT1Q3.py
    new file:   FMT1Q4.py
    new file:   FMT1Q5.py
    new file:   FMT1Q6.py
    new file:   FMT1Q7.py
    new file:   FMT1Q8.py
    new file:   README.md
```

IX. BRANCHING

A branch is an independent line of development. Branches are useful when developers want to work on a new feature/experiment without immediately changing the main version of the project. A developer can create a branch using git branch branch_name. The developer can then make changes and commit them without directly changing the main branch. Once completed, the branch can be merged into the main branch. Use git switch branch_name to select the branch or switch between branches.

X. MERGING BRANCHES

Merging branches combines changes from one branch into another. We can merge branches in two ways one by merge command and other by rebase command.

A. Merge

The git merge command combines two separate branches by creating a brand-new commit called a merge commit. It looks at the latest changes on both branches, blends them together into one updated project state, and saves that combined version as a new point in the project's history. This new merge commit is unique because it connects back to both of the original branches at the same time.

```

dhruv@Dhruvs-MacBook-Air Git_Task1 % git switch -c feature
Switched to a new branch 'feature'
dhruv@Dhruvs-MacBook-Air Git_Task1 % git branch
* feature
  main
dhruv@Dhruvs-MacBook-Air Git_Task1 % echo "Git merge">merge.txt
dhruv@Dhruvs-MacBook-Air Git_Task1 % git add.
git: 'add.' is not a git command. See 'git --help'.

The most similar command is
  add
dhruv@Dhruvs-MacBook-Air Git_Task1 % git add .
dhruv@Dhruvs-MacBook-Air Git_Task1 % git commit -m "Added merge"
[feature e7b9e5c] Added merge
 1 file changed, 1 insertion(+)
   create mode 100644 merge.txt
dhruv@Dhruvs-MacBook-Air Git_Task1 % git switch main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
dhruv@Dhruvs-MacBook-Air Git_Task1 % git merge feature
Updating e5eb6bd..e7b9e5c
Fast-forward
  merge.txt | 1 +
  1 file changed, 1 insertion(+)
   create mode 100644 merge.txt

```

B. Rebase

The git rebase command rewrites history to create a perfectly straight, linear timeline. Rather than combining two branches with an extra connecting commit, it detaches the unique commits from your branch and reapplies them sequentially on top of the latest version of the target branch. This makes it look like all the work was done in a

single, continuous line without any branches ever splitting off.

XI. RELATIVE REFERENCES

In Git, relative references like caret and tilde allow you to navigate backwards through your commit history from a specific point without needing to look up long commit ID hashes. Both symbols move your position upstream to a parent commit, but they navigate branching paths differently:

A. The Tilde (~)

It looks only at the primary, linear history. For example, `HEAD~1` is the immediate parent commit, and `HEAD~3` goes exactly three steps straight back, completely skipping over side branches.

B. The Caret (^)

This is essential after a merge commit, where a single commit has two different parents. For example, `HEAD^1` takes you back to the main branch you merged into, while `HEAD^2` takes you back onto the side branch that was brought in. Just using `HEAD^` takes you up one commit and similarly goes up the number of times you use `^` just like how tilde works if a number is not specified.

XII. REVERSING CHANGES

In Git, reversing changes depends entirely on whether you want to cleanly rewrite history or safely preserve it. For a report, this is best explained by contrasting the two primary commands used to undo commits:

A. Git reset

`git reset` rewrites history by moving the branch pointer backward in time. It essentially deletes commits from the current timeline as if they never happened.

B. Git revert

While git reset works great for local branches, it doesn't work for remote branches that others are using. `git revert` preserves history by creating a new, opposing commit. Instead of erasing a bad commit, it creates a new reversal commit that acts like an undo layer. It leaves the original bad commit visible in the timeline while adding a new point that cancels out its changes.

XIII. MOVING COMMITS AROUND

A. Cherry picking

Cherry picking is a simple and straightforward way to copy a bunch of commits onto another branch. Select the branch you want the copied commits to go to and then use the command `git cherrypick <commit hash>`.

B. Interactive rebasing

Interactive rebasing, executed via `git rebase -i`, provides a mechanism to modify a branch's commit history before integration. This opens a graphical interface where we can reorder commits and select which commits to drop.

XIV. UNDOING

If you stage a file you didn't mean to or start an experiment you want to discard, we can undo these changes using `git restore`. There are two ways for this:

A. Unstaging files

`git restore -staged <file>` moves it out of the staging area.

B. Discarding files

`git restore <file>` discards your edits to the file and removes it from the working directory.

XV. TAGS

We can also mark important events/ merges/ changes using tags. This is a way to permanently mark historical points in your projects history. We can assign tags using `git tag tag_name <commit hash>`.

XVI. DESCRIBE

You can see your position relative to the closest tag

using `git describe`

XIV. REMOTE REPOSITORIES

A remote repository is simply a copy of your project's code stored online or on an external server separate from the developers local machine. It acts as a central hub where everyone on a team can save their work, track changes securely, and parallelly collaborate efficiently.

A. Cloning

Cloning (`git clone`) is the process of downloading a complete copy of a remote repository onto your local computer.

```
Last login: Sat Sep 5 14:34:29 on tty000
dhruv@Dhruvs-MacBook-Air ~ % cd ~/Desktop
dhruv@Dhruvs-MacBook-Air Desktop % git clone https://github.com/dhruv270608/-Dhruv-sunil-Driverless_Taskphase
Cloning into '-Dhruv-sunil-Driverless_Taskphase'...
remote: Enumerating objects: 15, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 13 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 4.90 KiB | 4.90 MiB/s, done.
dhruv@Dhruvs-MacBook-Air Desktop % cd ~/Desktop/-Dhruv-sunil-Driverless_Taskphase
dhruv@Dhruvs-MacBook-Air ~-Dhruv-sunil-Driverless_Taskphase % ls
FMT101.py  FMT103.py  FMT105.py  FMT107.py  README.md
FMT102.py  FMT104.py  FMT106.py  FMT108.py
dhruv@Dhruvs-MacBook-Air ~-Dhruv-sunil-Driverless_Taskphase % cd ~/Desktop
dhruv@Dhruvs-MacBook-Air Desktop % git clone https://github.com/dhruv270608/Git_Task1
Cloning into 'Git_Task1'...
warning: You appear to have cloned an empty repository.
dhruv@Dhruvs-MacBook-Air Desktop % cd ~/Desktop/Git_Task1
```

B. Remote branches

Remote branches reflect the state if remote repositories (since you last talked to those remote repositories). They help you understand the difference between your local work and public work. These pointers are read-only and cannot be directly manipulated or checked out by the developer.

C. *Fetching* `git fetch` performs two steps. It downloads commits that the remote repository has but are missing from the local repository and updates where are remote repository points.

D. *Pulling* `git pull` is a multistep process that fetches the missing commits and then integrates the remote repository updates onto the local repository through merge or rebase(`git pull --rebase`).

```
dhruv@Dhruvs-MacBook-Air Git_Task1 % git pull
remote: Enumerating objects: 5, done.
remote: Counting objects: 100% (5/5), done.
remote: Compressing objects: 100% (3/3), done.
remote: Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (3/3), 993 bytes | 248.00 KiB/s, done.
From https://github.com/dhruv270608/Git_Task1
 72ee2b5..e55cb6d  main       -> origin/main
Updating 72ee2b5..e55cb6d
Fast-forward
 FMT101.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

E. Pushing

Just as how `git pull` fetches work from remote we can upload work to remote using `git push` which is accessible to everyone.

```
Last login: Sat Sep 5 14:34:29 on tty000
dhruv@Dhruvs-MacBook-Air ~ % cd ~/Desktop
dhruv@Dhruvs-MacBook-Air Desktop % git clone https://github.com/dhruv270608/-Dhruv-sunil-Driverless_Taskphase
Cloning into '-Dhruv-sunil-Driverless_Taskphase'...
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 13 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (13/13), 4.90 KiB | 4.90 MiB/s, done.
dhruv@Dhruvs-MacBook-Air Desktop % cd ~/Desktop/-Dhruv-sunil-Driverless_Taskphase
dhruv@Dhruvs-MacBook-Air ~-Dhruv-sunil-Driverless_Taskphase % ls
FMT101.py  FMT103.py  FMT105.py  FMT107.py  README.md
FMT102.py  FMT104.py  FMT106.py  FMT108.py
dhruv@Dhruvs-MacBook-Air ~-Dhruv-sunil-Driverless_Taskphase % cd ~/Desktop
dhruv@Dhruvs-MacBook-Air Desktop % git clone https://github.com/dhruv270608/Git_Task1
Cloning into 'Git_Task1'...
warning: You appear to have cloned an empty repository.
dhruv@Dhruvs-MacBook-Air Desktop % cd ~/Desktop/Git_Task1
```

F. Remote tracking

Connection between main and o/main is explained simply by remote tracking property of branch. The main branch is set to track o/main this means there is an implied merge target and implied push destination for the main branch. Which means instead of `git pull origin main` using `git pull` works the same as because your branch is tracking o/main, `git` already knows your target and similarly for push. To create your own tracker use: `git checkout -b tracker_name o/main`.

ACKNOWLEDGMENTS

I would like to express my sincere gratitude to Formula Manipal for providing the opportunity to complete this comprehensive task on Git version control. Working through the repository requirements significantly enhanced my practical understanding of distributed version control, branching strategies, and collaborative code management workflows in an engineering context and how this is used in the real world. Additionally, I thank the Manipal Institute of Technology (MIT), Manipal for providing the computing environment and academic platform that facilitated this skillbuilding exercise.

REFERENCES

[1] [Learn git braching](#)